

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет
ИТМО»

Факультет программной инженерии и компьютерной техники

Основы программной инженерии
Лабораторная работа № 4
Вариант: 7396

Выполнили:

Вильданов Ильнур Наилевич
Бондарев Алексей Михайлович
Группа №Р3212

Преподаватель:

Лазеев Сергей Максимович

Санкт-Петербург
2025 г.

Оглавление

<i>Текст задания</i>	3
<i>Задание №1</i>	4
<i>Задание №2</i>	7
<i>Задание №3</i>	9
<i>Задание №4</i>	11
<i>Вывод</i>	17

Текст задания

Лабораторная работа #4

Вариант 7396

Внимание! У разных вариантов разный текст задания!

1. Для своей программы из [лабораторной работы #3](#) по дисциплине "Веб-программирование" реализовать:

- MBeap, считающий общее число установленных пользователем точек, а также число точек, не попадающих в область. В случае, если пользователь совершил 3 "промаха" подряд, разработанный MBeap должен отправлять оповещение об этом событии.
- MBeap, определяющий средний интервал между кликами пользователя по координатной плоскости.

2. С помощью утилиты JConsole провести мониторинг программы:

- Снять показания MBeap-классов, разработанных в ходе выполнения задания 1.
- Определить наименование и версию JVM, поставщика виртуальной машины Java и номер её сборки.

3. С помощью утилиты VisualVM провести мониторинг и профилирование программы:

- Снять график изменения показаний MBeap-классов, разработанных в ходе выполнения задания 1, с течением времени.
- Определить имя потока, потребляющего наибольший процент времени CPU.

4. С помощью утилиты VisualVM и профилировщика IDE NetBeans, Eclipse или Idea локализовать и устранить проблемы с производительностью в [программе](#). По результатам локализации и устранения проблемы необходимо составить отчёт, в котором должна содержаться следующая информация:

- Описание выявленной проблемы.
- Описание путей устранения выявленной проблемы.
- Подробное (со скриншотами) описание алгоритма действий, который позволил выявить и локализовать проблему.

Студент должен обеспечить возможность воспроизведения процесса поиска и локализации проблемы по требованию преподавателя.

Задание №1

Исходный код MBean классов:

```
package ru.itmo.app.mbean;

import java.io.Serializable;
import java.util.concurrent.atomic.AtomicInteger;
import javax.management.MBeanNotificationInfo;
import javax.management.Notification;
import javax.management.NotificationBroadcasterSupport;

import jakarta.annotation.PostConstruct;
import jakarta.enterprise.context.ApplicationScoped;
import jakarta.inject.Named;

@Named
@ApplicationScoped
public class PointStatisticsMBean
    extends NotificationBroadcasterSupport
    implements PointStatisticsMXBean, Serializable {

    private static final int MISS_THRESHOLD = 3;

    private final AtomicInteger totalPoints = new AtomicInteger(0);
    private final AtomicInteger pointsNotInArea = new AtomicInteger(0);

    private int consecutiveMisses = 0;
    private long sequenceNumber = 1;

    @PostConstruct
    public void init() { }

    @Override
    public int getTotalPoints() {
        return totalPoints.get();
    }

    @Override
    public int getPointsNotInArea() {
        return pointsNotInArea.get();
    }

    @Override
    public int getConsecutiveMisses() {
        return consecutiveMisses;
    }

    public void addPoint(double x, double y, double r, boolean inArea) {
        totalPoints.incrementAndGet();

        if (!inArea) {
            pointsNotInArea.incrementAndGet();
            consecutiveMisses++;

            if (consecutiveMisses > 0 && consecutiveMisses % MISS_THRESHOLD
== 0) {
                Notification notif = new Notification(
                    "ConsecutiveMisses",
                    this.getClass().getName(),
                    sequenceNumber++,
                    System.currentTimeMillis(),
                    "Reached " + consecutiveMisses + " consecutive
misses"

                );
            }
        }
    }
}
```

```

        notif.setUserData("Number of consecutive misses: " +
consecutiveMisses);
        sendNotification(notif);
    }
    } else {
        consecutiveMisses = 0;
    }
}

@Override
public MBeanNotificationInfo[] getNotificationInfo() {
    String[] types = new String[] { "ConsecutiveMisses" };
    String name = Notification.class.getName();
    String description = "Notification about consecutive misses";
    MBeanNotificationInfo info = new MBeanNotificationInfo(types, name,
description);
    return new MBeanNotificationInfo[] { info };
}
}

```

```

package ru.itmo.app.mbean;

public interface PointStatisticsMXBean {
    int getTotalPoints();
    int getPointsNotInArea();
    int getConsecutiveMisses();
}

```

```

@Named
@ApplicationScoped
public class ClickIntervalMBean implements ClickIntervalMXBean, Serializable
{
    private static final int MAX_INTERVALS = 100;
    private final Queue<Long> clickTimestamps = new LinkedList<>();
    private final Queue<Long> intervals = new LinkedList<>();
    private long totalIntervalTime = 0;
    private long lastClickTime = 0;
    private int intervalCount = 0;

    @PostConstruct
    public void init() {
    }

    @Override
    public double getAverageInterval() {
        if (intervalCount == 0) {
            return 0;
        }
        return (double) totalIntervalTime / intervalCount;
    }

    @Override
    public long getLastInterval() {
        if (intervals.isEmpty()) {
            return 0;
        }
        return intervals.peek();
    }

    @Override
    public int getClickCount() {
    }
}

```

```

        return clickTimestamps.size();
    }

    @Override
    public int getIntervalCount() {
        return intervalCount;
    }

    public void registerClick() {
        long currentTime = System.currentTimeMillis();

        clickTimestamps.add(currentTime);
        if (clickTimestamps.size() > MAX_INTERVALS + 1) {
            clickTimestamps.poll();
        }

        if (lastClickTime > 0) {
            long interval = currentTime - lastClickTime;
            intervals.add(interval);
            totalIntervalTime += interval;
            intervalCount++;

            if (intervals.size() > MAX_INTERVALS) {
                long oldestInterval = intervals.poll();
                totalIntervalTime -= oldestInterval;
                intervalCount--;
            }
        }

        lastClickTime = currentTime;
    }
}

```

```

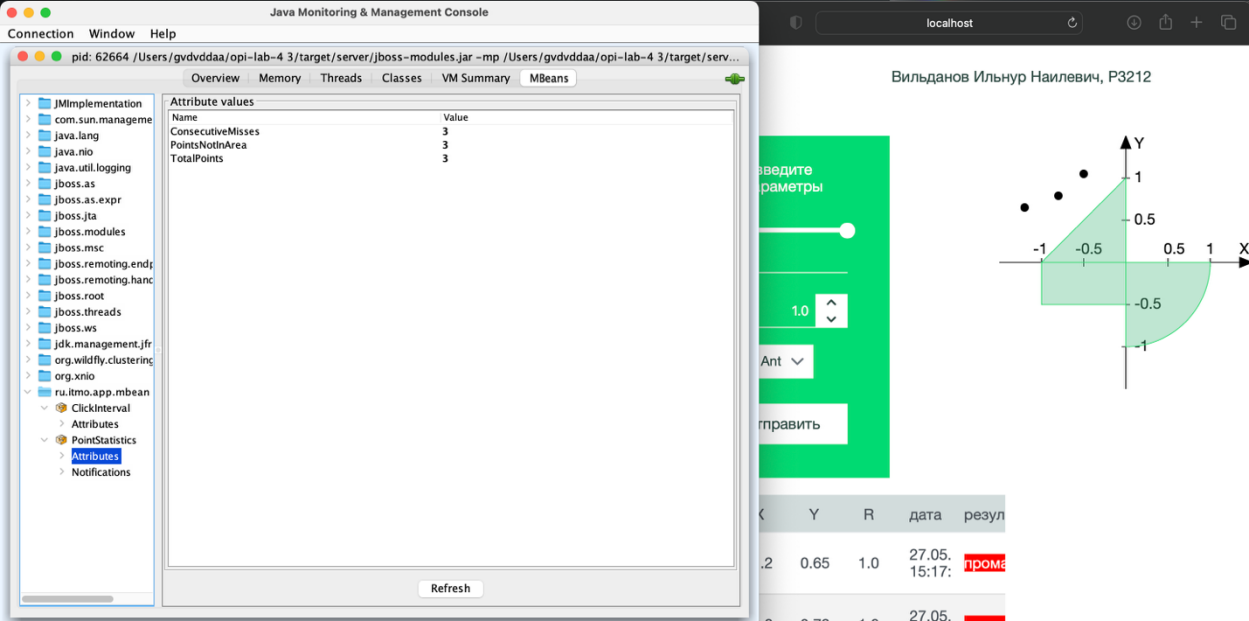
package ru.itmo.app.mbean;

public interface ClickIntervalMXBean {
    double getAverageInterval();
    long getLastInterval();
    int getClickCount();
    int getIntervalCount();
}

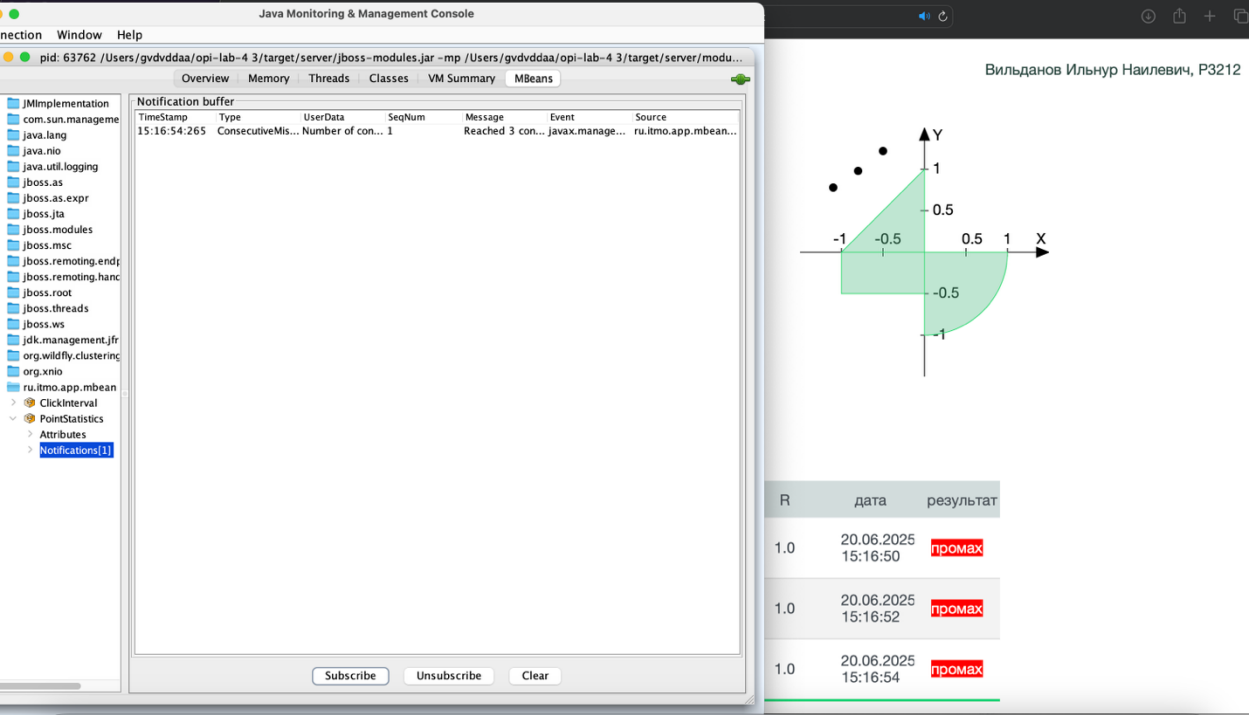
```

Задание №2

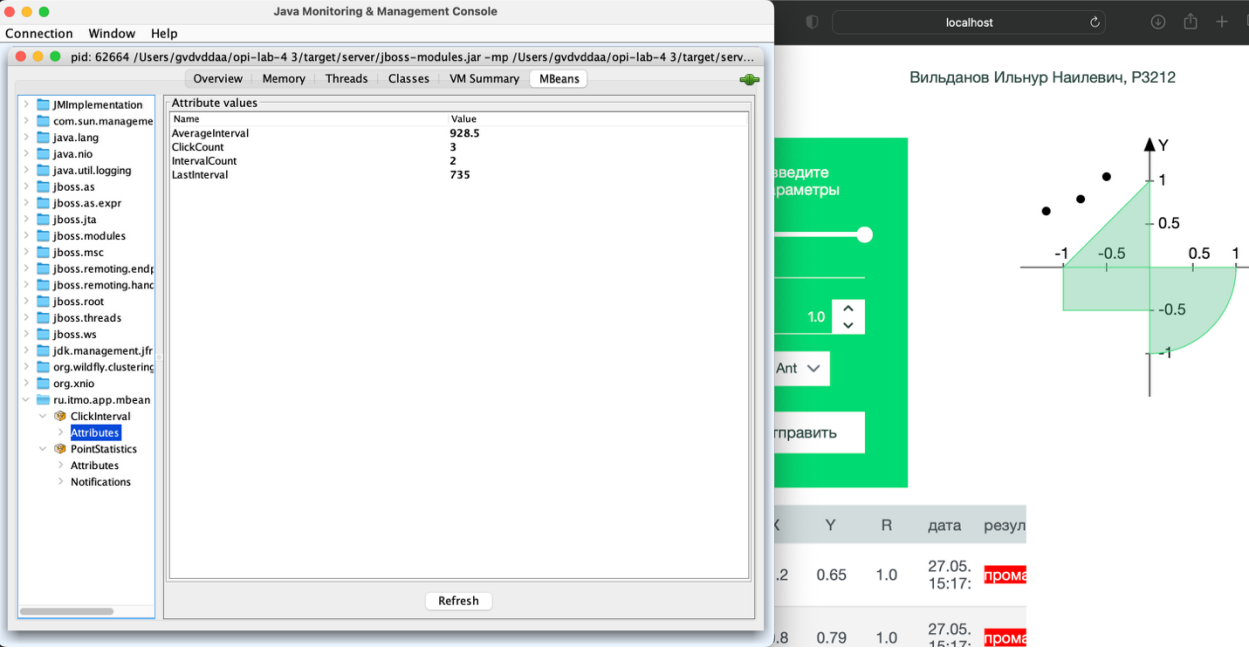
Показания MBean-классов, разработанных в задании №1:
PointStatisticsMBean.java



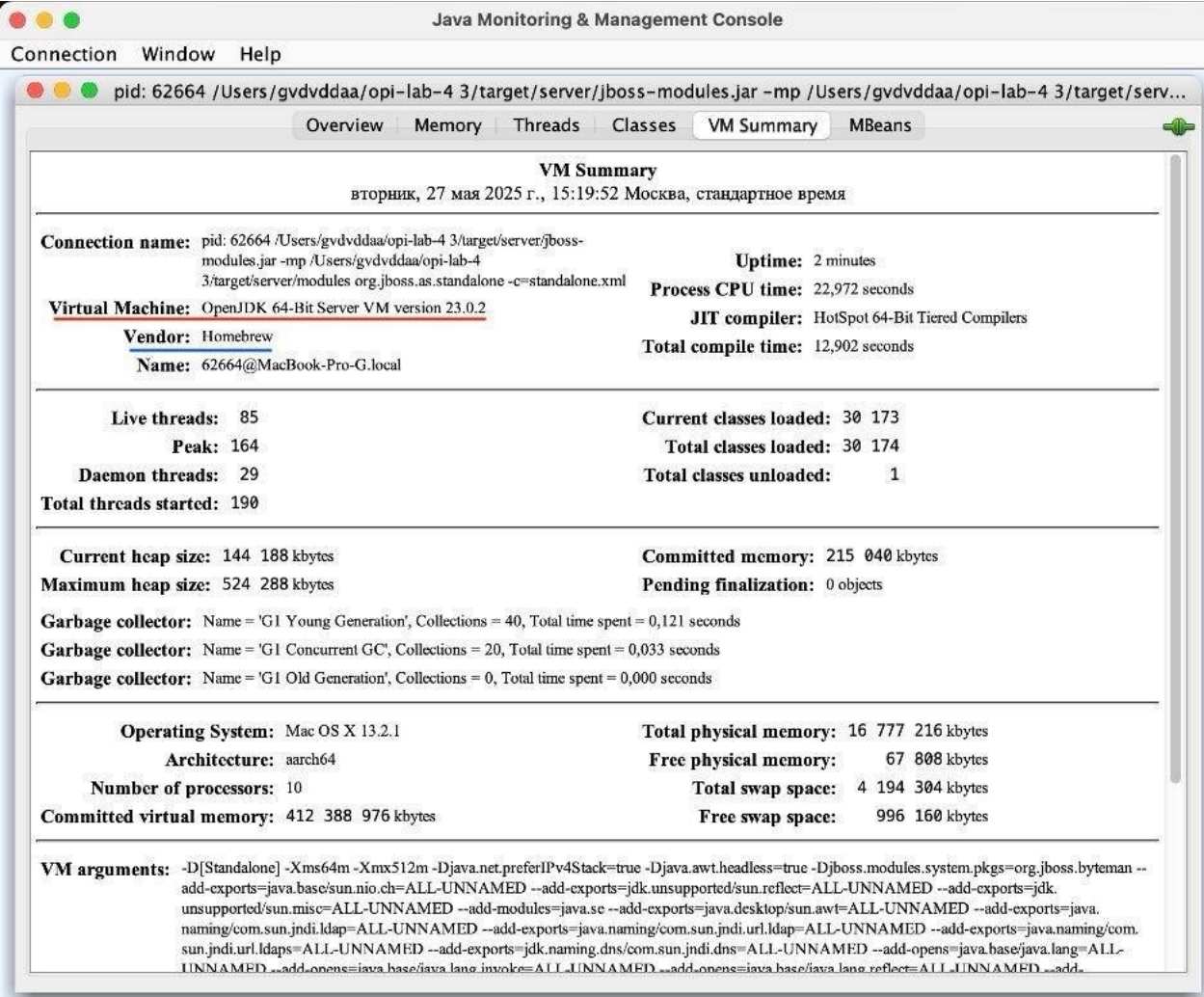
В случае трёх промахов подряд приходит оповещение об этом событии:



ClickIntervalMBean.java

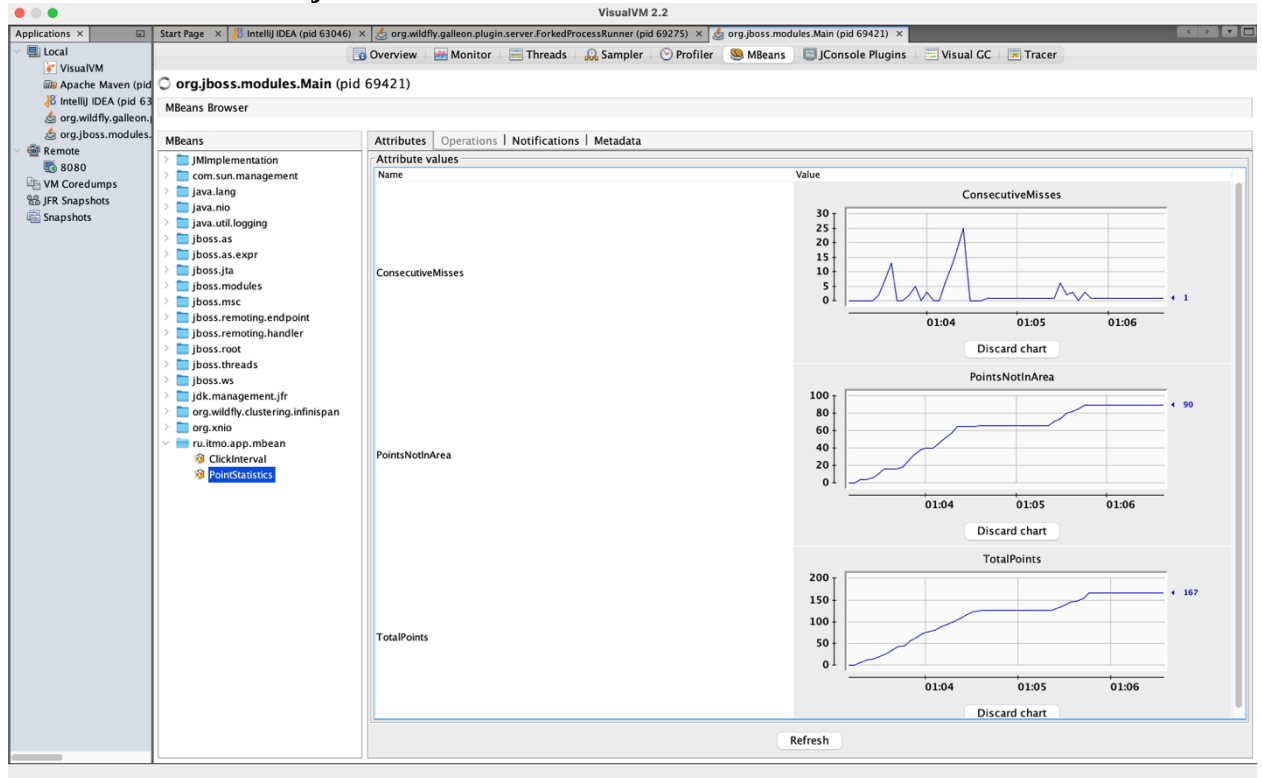


Наименование и версия JVM, поставщик виртуальной машины Java и номер ее сборки:

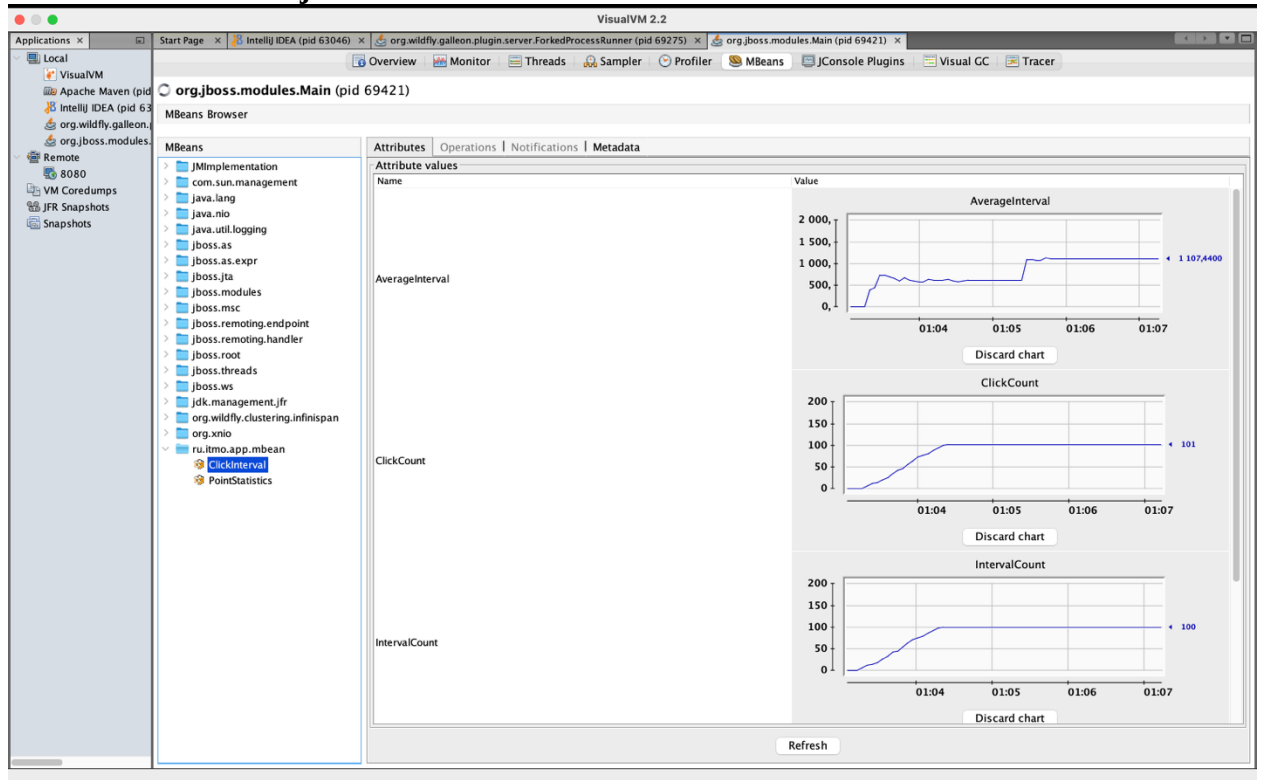


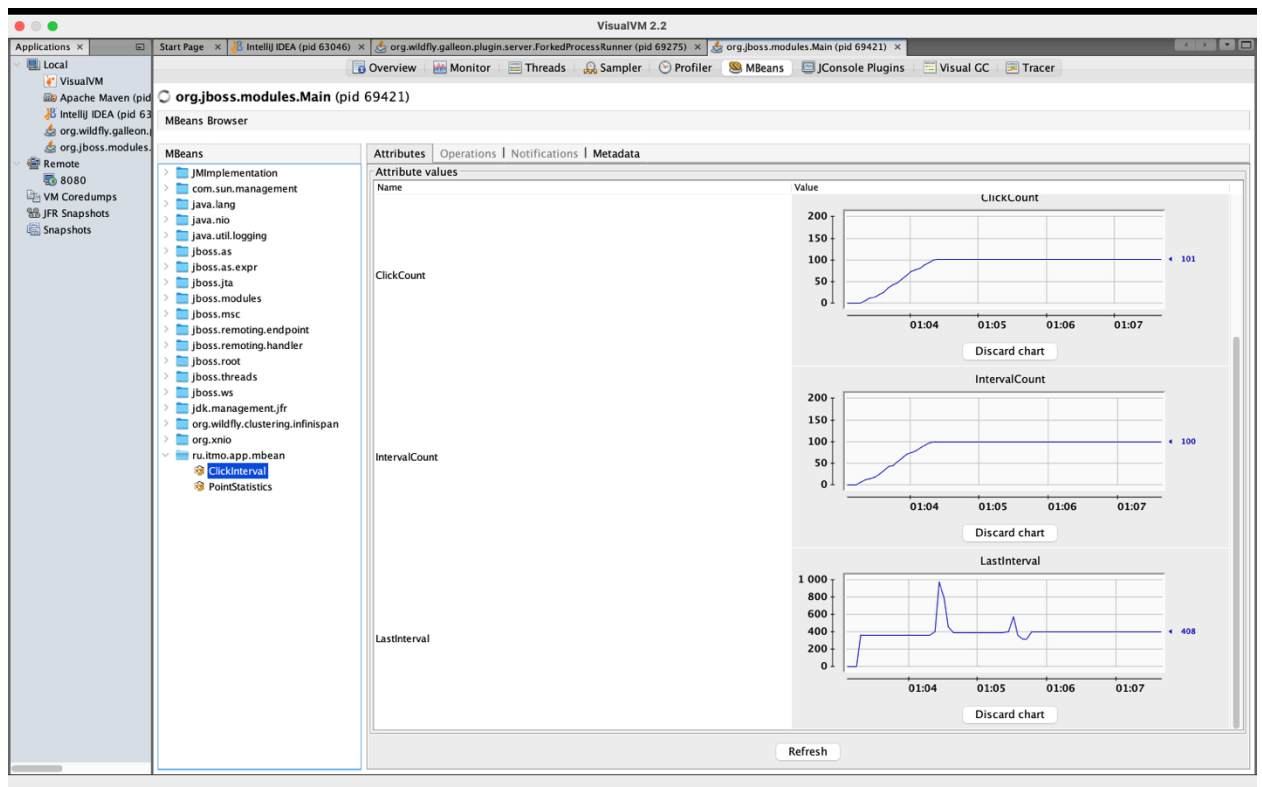
Графики изменения показаний МВеп-классов:

PointStatisticsMBean.java



ClickIntervalMBean.java





Поток, потребляющий наибольший процент времени CPU:

VisualVM 2.2

org.jboss.modules.Main (pid 63707)

Sampler

Sample: CPU Memory Stop

Status: CPU sampling in progress

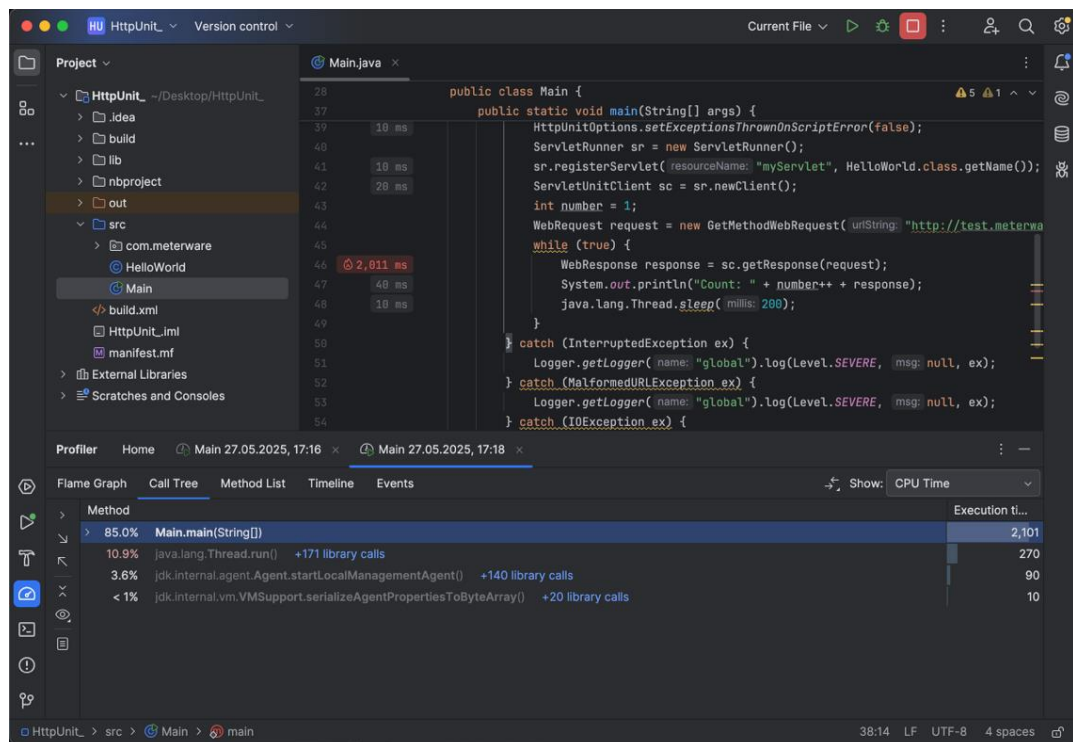
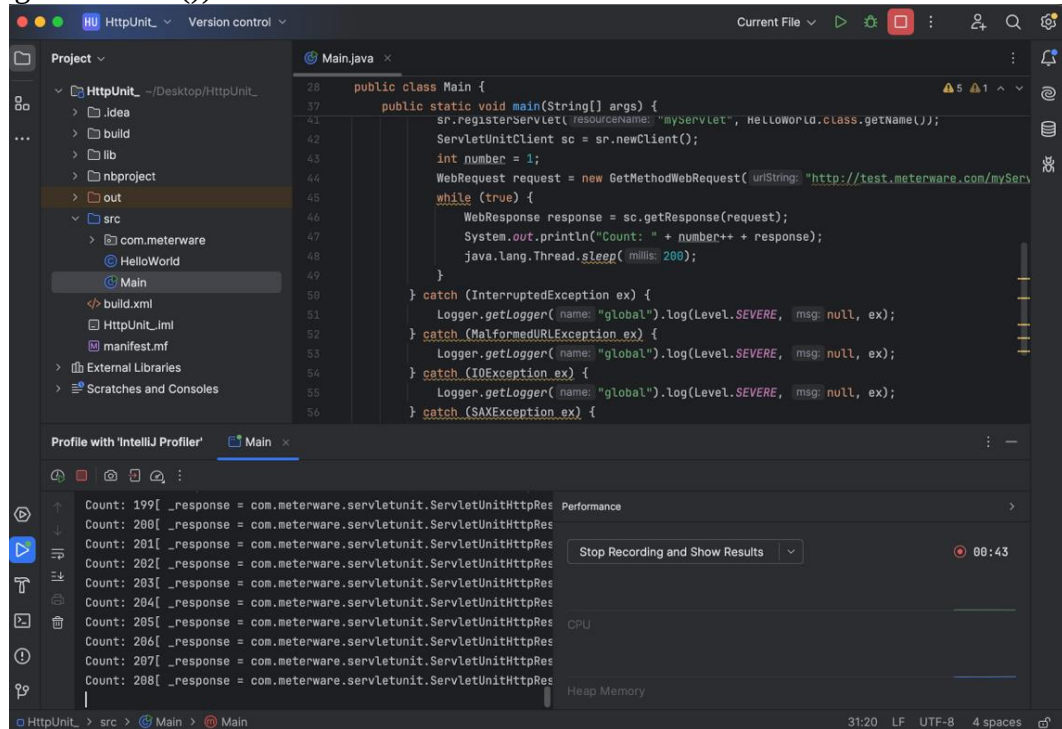
CPU samples Thread CPU time

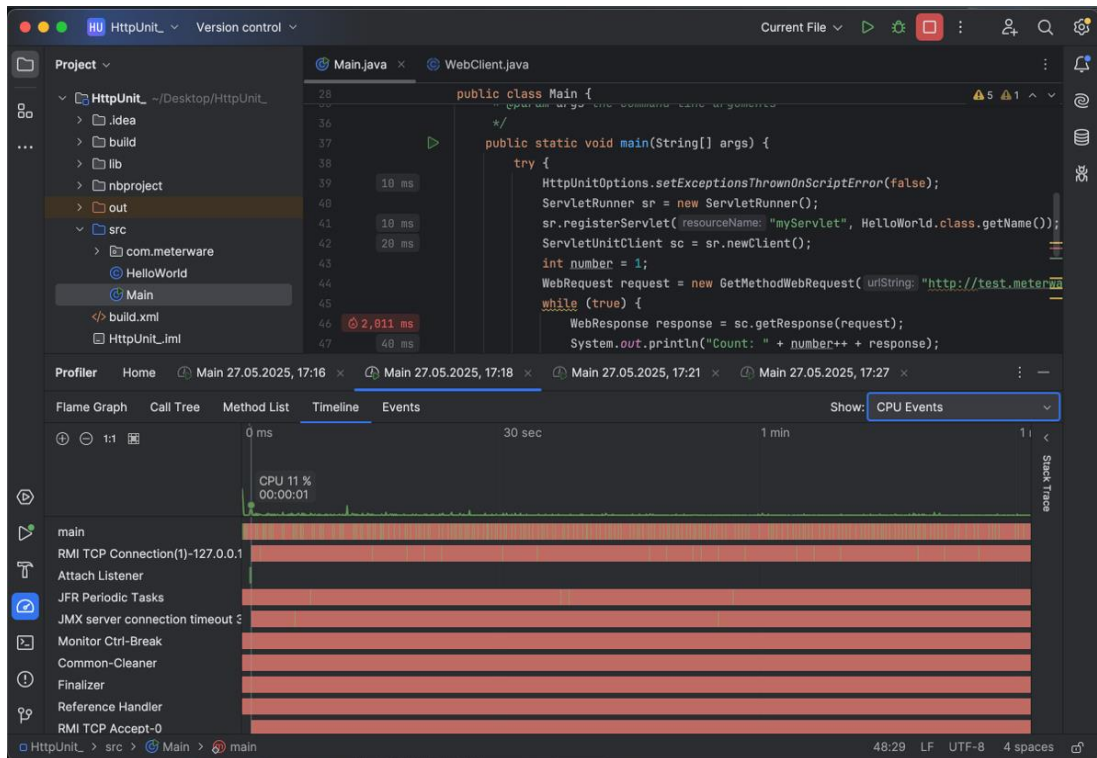
Results: Statistics: Threads Count: 88 Total Time (CPU): 21 726 ms Thread Dump

Name	Thread Time (CPU)	Thread Time (CPU) / sec
default task-1	7 119 ms (32,8 %)	0,0 ms (0 %)
RMI TCP Connection(102)-127.0.0.1	4 632 ms (21,3 %)	0,0 ms (0 %)
RMI TCP Connection(104)-127.0.0.1	4 250 ms (19,6 %)	0,557 ms (0,1 %)
MSC service thread 1-3	1 215 ms (5,6 %)	0,0 ms (0 %)
RMI TCP Connection(103)-127.0.0.1	927 ms (4,3 %)	60,9 ms (6,1 %)
ServerService Thread Pool -- 33	491 ms (2,3 %)	0,0 ms (0 %)
DestroyJavaVM	483 ms (2,2 %)	0,0 ms (0 %)
MSC service thread 1-1	443 ms (2 %)	0,0 ms (0 %)
MSC service thread 1-8	287 ms (1,3 %)	0,0 ms (0 %)
MSC service thread 1-5	266 ms (1,2 %)	0,0 ms (0 %)
MSC service thread 1-6	253 ms (1,2 %)	0,0 ms (0 %)
FileSystemWatcher	221 ms (1 %)	1,99 ms (0,2 %)
default I/O-17	206 ms (1 %)	0,0 ms (0 %)
DeploymentScanner-threads - 2	150 ms (0,7 %)	3,19 ms (0,3 %)
MSC service thread 1-4	121 ms (0,6 %)	0,0 ms (0 %)
MSC service thread 1-7	103 ms (0,5 %)	0,0 ms (0 %)
MSC service thread 1-2	92,3 ms (0,4 %)	0,0 ms (0 %)
Attach Listener	74,0 ms (0,3 %)	0,0 ms (0 %)
default I/O-7	56,1 ms (0,3 %)	0,0 ms (0 %)
management I/O-2	55,8 ms (0,3 %)	0,0 ms (0 %)
management-handler-thread - 1	31,7 ms (0,1 %)	0,0 ms (0 %)
Notification Thread	23,8 ms (0,1 %)	0,0 ms (0 %)
RMI TCP Accept-0	18,9 ms (0,1 %)	0,0 ms (0 %)
management task-1	16,8 ms (0,1 %)	0,0 ms (0 %)
Weld Thread Pool -- 11	15,3 ms (0,1 %)	0,0 ms (0 %)
Weld Thread Pool -- 2	15,3 ms (0,1 %)	0,0 ms (0 %)

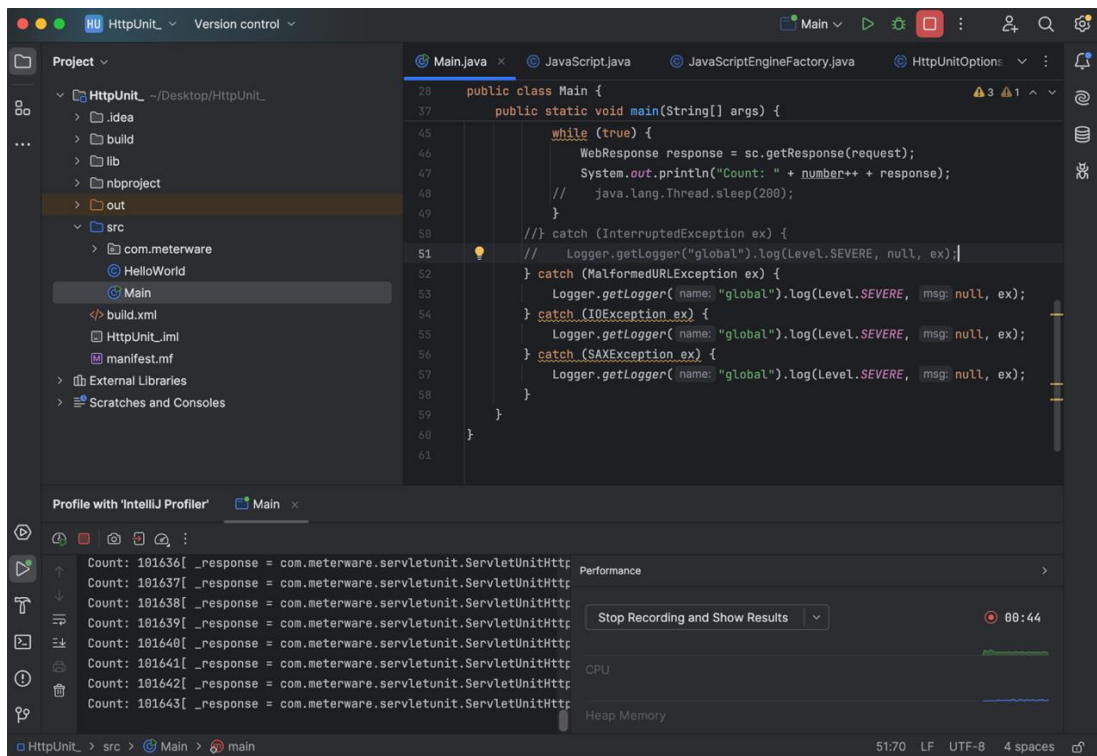
Задание №4

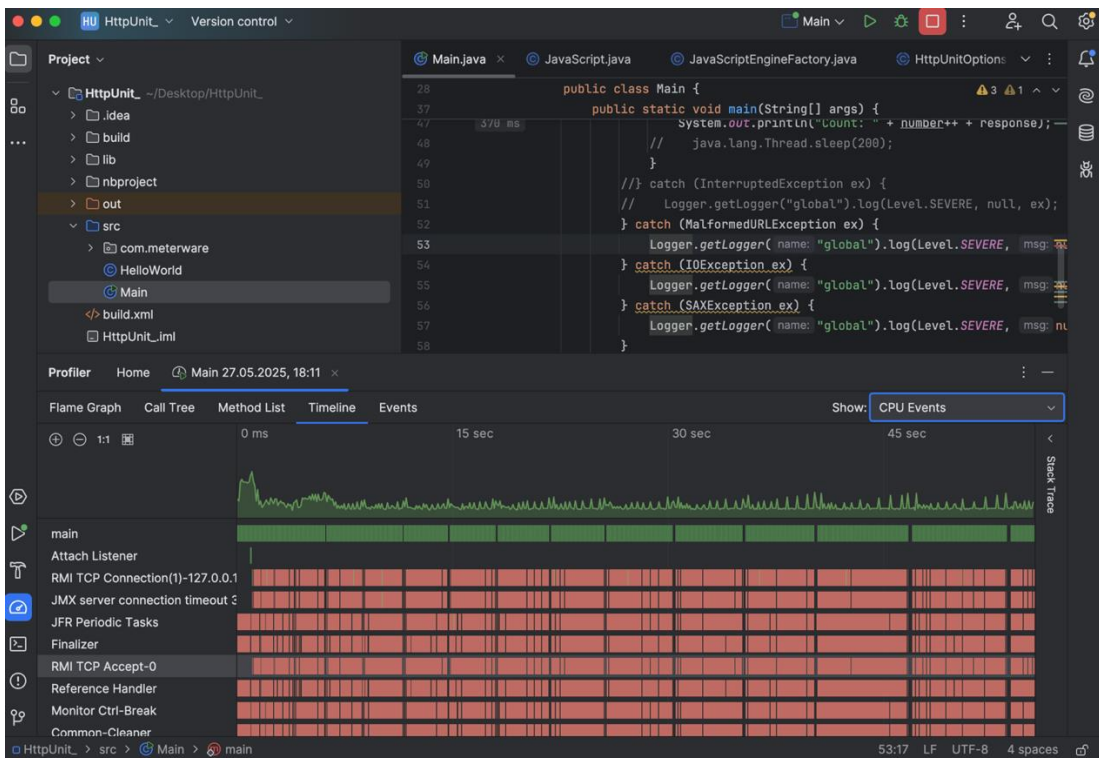
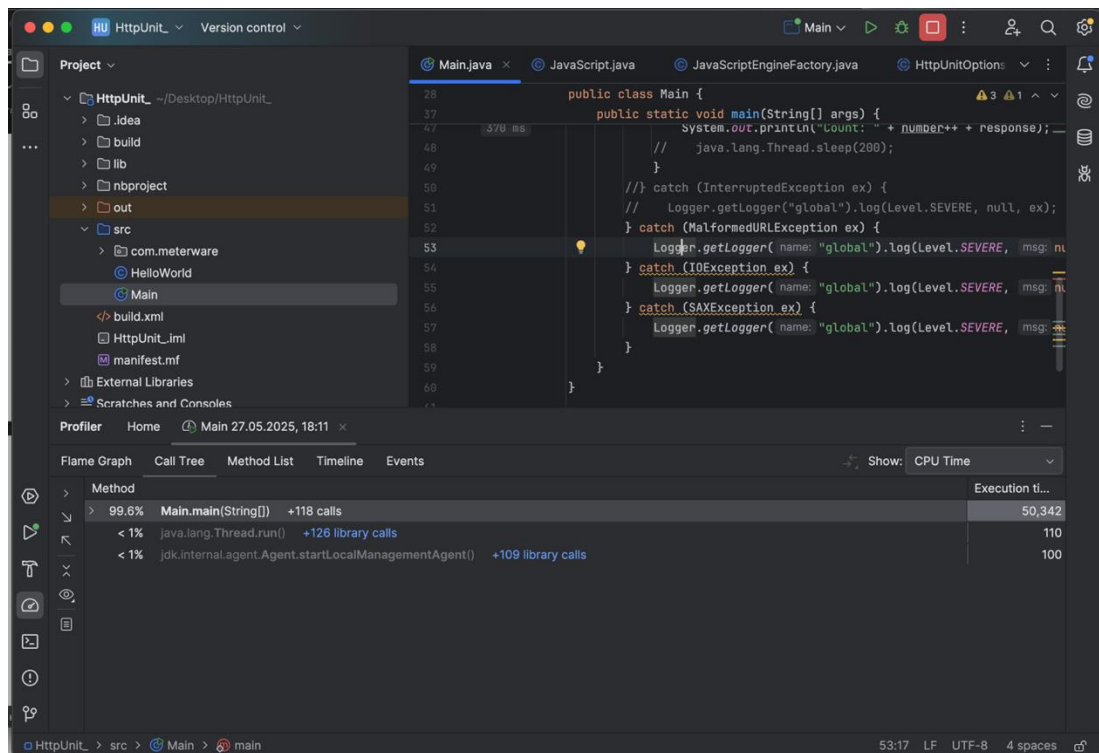
При анализе производительности программы с помощью профилировщика IntelliJ IDEA была обнаружена проблема, связанная с избыточным вызовом метода `java.lang.Thread.sleep(200)`. Данный метод вызывает приостановку выполнения потока на 200 миллисекунд, что приводит к задержкам при работе программы (main процесс большую часть времени находится в ожидании) и снижению общей производительности (всего 208 counts за 43 секунды и 10,9% работы CPU направлены на выполнение `java.lang.Thread.run()`).





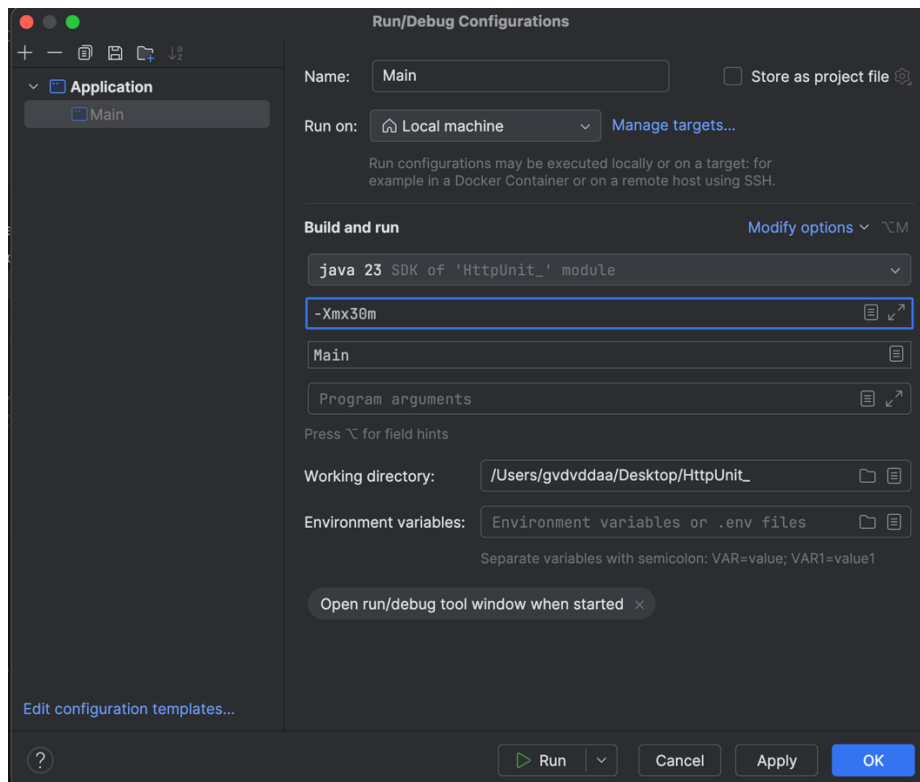
Метод не несёт никакой функциональной нагрузки и является избыточным. Его удаление не повлияло на корректность работы приложения, при этом main процесс стал занят большую часть времени, а также удалось добиться серьёзного повышения производительности за счёт устранения задержек (101640 counts за 43 секунды, 99,6 % работы CPU направлен на выполнение Main.main(String [])).



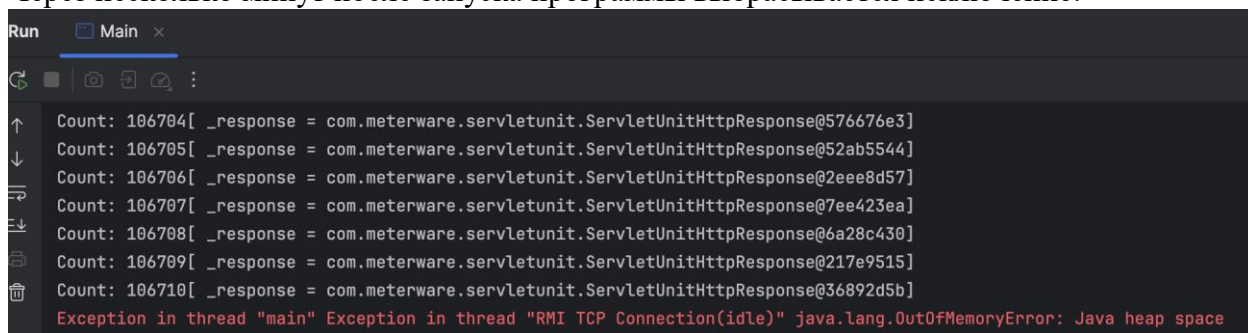


Утечка памяти:

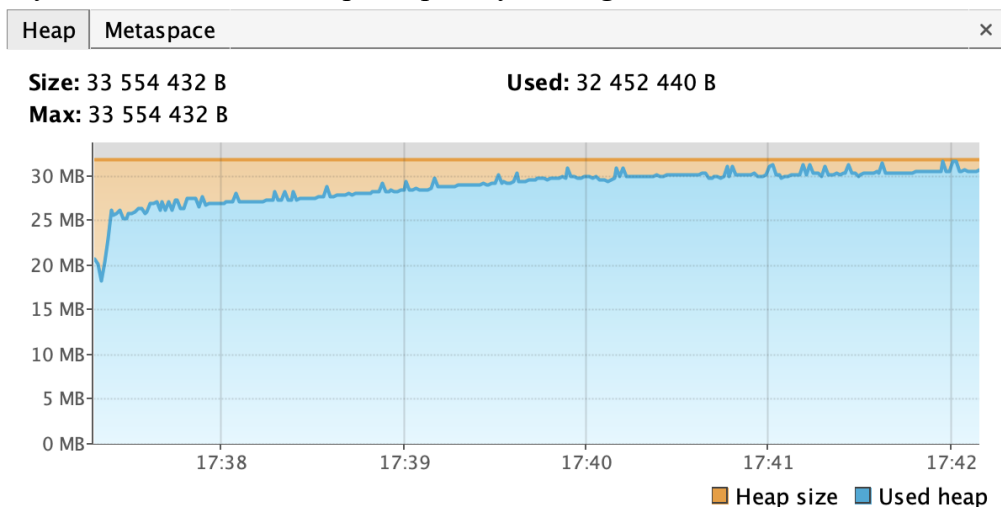
Для начала установим ограничение максимального размера кучи на 30 Мб (Для этого в конфигурацию запуска добавим опцию (-Xmx30m) для VM):



Через несколько минут после запуска программы выбрасывается исключение:

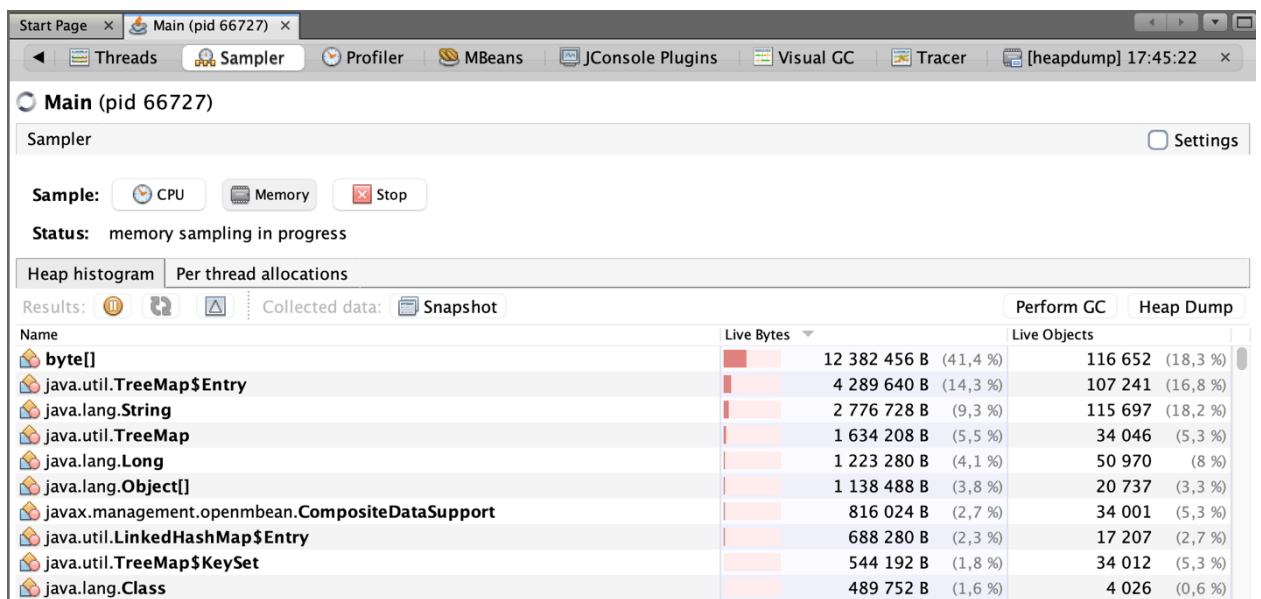


При анализе графика использования памяти в куче видно, что размер кучи постоянно увеличивается, несмотря на работу Garbage Collector'a:



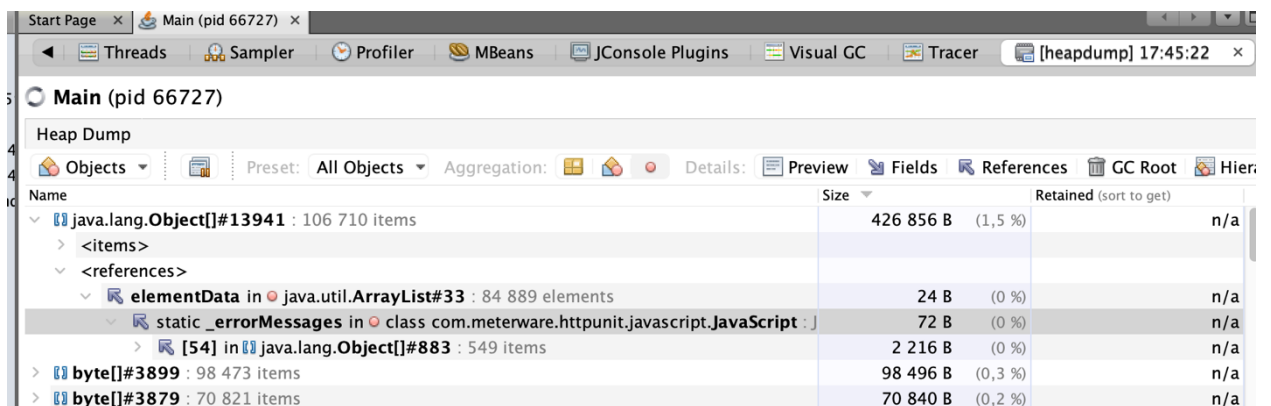
Следовательно, можно сделать вывод, что утечка памяти действительно существует.

Локализуем проблему. Для этого проанализируем кучу и найдем классы объектов, которые занимают много памяти:



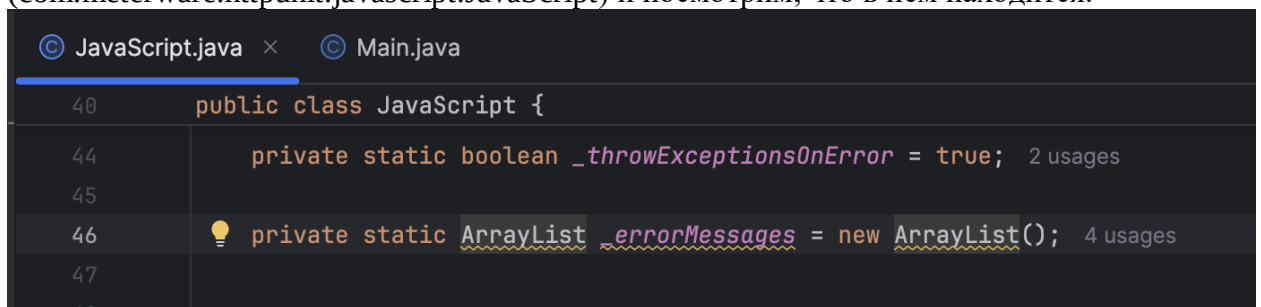
Name	Live Bytes	Live Objects
byte[]	12 382 456 B (41,4 %)	116 652 (18,3 %)
java.util.TreeMap\$Entry	4 289 640 B (14,3 %)	107 241 (16,8 %)
java.lang.String	2 776 728 B (9,3 %)	115 697 (18,2 %)
java.util.TreeMap	1 634 208 B (5,5 %)	34 046 (5,3 %)
java.lang.Long	1 223 280 B (4,1 %)	50 970 (8 %)
java.lang.Object[]	1 138 488 B (3,8 %)	20 737 (3,3 %)
javax.management.openmbean.CompositeDataSupport	816 024 B (2,7 %)	34 001 (5,3 %)
java.util.LinkedHashMap\$Entry	688 280 B (2,3 %)	17 207 (2,7 %)
java.util.TreeMap\$KeySet	544 192 B (1,8 %)	34 012 (5,3 %)
java.lang.Class	489 752 B (1,6 %)	4 026 (0,6 %)

Больше всего у нас объектов типа byte[] (массивов байт), а также объектов String, которые включают в себя объекты типа byte[]. При анализе Heap Dump наибольший размер в памяти занял объект java.lang.Object[]#13941 - arrayList под названием _errorMessages, в котором содержится очень большое количество элементов типа String с информацией об ошибках.



Name	Size	Retained (sort to get)
java.lang.Object[]#13941 : 106 710 items	426 856 B (1,5 %)	n/a
> <items>		
> <references>		
-> elementData in java.util.ArrayList#33 : 84 889 elements	24 B (0 %)	n/a
-> static _errorMessages in class com.meterware.httpunit.javascript.JavaScript : J	72 B (0 %)	n/a
-> [54] in java.lang.Object[]#883 : 549 items	2 216 B (0 %)	n/a
> byte[]#3899 : 98 473 items	98 496 B (0,3 %)	n/a
> byte[]#3879 : 70 821 items	70 840 B (0,2 %)	n/a

Найдём исходный класс по его расположению (com.meterware.httpunit.javascript.JavaScript) и посмотрим, что в нём находится.



```
public class JavaScript {  
    private static boolean _throwExceptionsOnError = true; 2 usages  
    private static ArrayList _errorMessages = new ArrayList(); 4 usages  
}
```

Добавление элементов в данный список происходит в методе класса JavaScript, а вот метод очищения списка нигде применяется:

```

202         throw new ScriptException( errorMessage );
203     } else {
204         errorMessages.add( errorMessage );
205     }
206 }

```

Метод `clearErrorMessages` из класса `JavaScript` используется в классе `com.meterware.httpunit.javascript.JavaScriptEngineFactory`:

```

59         > static void clearErrorMessages() { errorMessages.clear(); }

```

Метод `getErrorMessages` из `JavaScriptEngineFactory` используется в классе `com.meterware.httpunit.HttpUnitOptions`:

```

78         public String[] getErrorMessages() { 1 usage
79             return JavaScript.getErrorMessages();
80         }

```

А вот метод `clearScriptErrorMessages` из `HttpUnitOptions` не используется нигде:

```

456         public static void clearScriptErrorMessages() { no usages
457             getScriptingEngine().clearErrorMessages();
458         }

```

Элементы накапливаются в списке `_errorMessages` при этом нигде не очищаются, что приводит к утечке памяти в связи с отсутствием очистки.

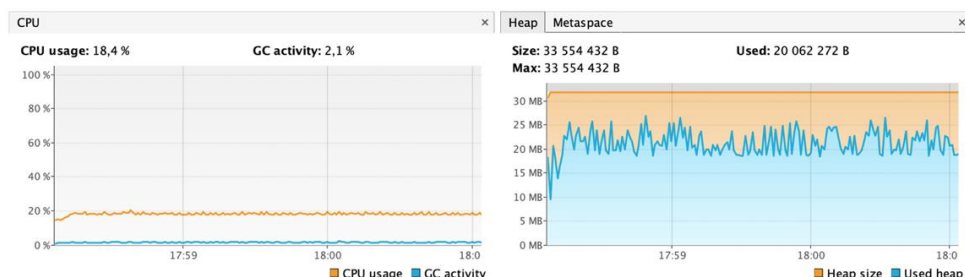
Устраним данную проблему, добавив очистку списка `_errorMessages` после выполнения каждого запроса (методом `clearScriptErrorMessages`) в главном цикле программы:

```

45         while (true) {
46             WebResponse response = sc.getResponse(request);
47             System.out.println("Count: " + number++ + response);
48             HttpUnitOptions.clearScriptErrorMessages();
49         }

```

Проверим устранение утечки памяти. Для этого снова запустим программу и проведём анализ графика изменения использования памяти в куче. В результате наблюдаем, что теперь не расходует больше 30 Мб кучи, размер кучи постоянно не растёт и сборщик мусора работает в нормальном режиме:



Следовательно, утечка памяти была успешно устранена.

Вывод

Во время выполнения лабораторной работы мы познакомились с практикой написания MBean в веб-приложениях, были изучены утилиты для мониторинга и профилирования работы программы JConsole и VisualVM, а также был получен опыт по полученным данным определять утечки памяти и устранять их.